```
            if a[mid] < target <= a[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1, c

n = 10 + SEED_DIGIT
original = [100 + ID_SUM + i * NAME_LENGTH for i in range(n)]
k = SEED_DIGIT
rotated = original[k:] + original[:k]
target = original[(ID_SUM + ARTIST_SUM) % n]
missing = target + 1

print("Original:", original)
print("Rotation point k:", k)
print("Rotated :", rotated)
print("Target  :", target)
for label, t in (("Target", target), ("Missing target", missing)):
    bi, bc = rotated_binary_search(rotated, t)
    li, lc = linear_search(rotated, t)
    print(f"{label} {t}: Modified binary -> index={bi}, comparisons={bc} | Linear -> index={li}, comparisons={lc}")
```

[6] Python

```
Original: [124, 130, 136, 142, 148, 154, 160, 166, 172, 178, 184, 190, 196, 202, 208, 214, 220]
Rotation point k: 7
Rotated : [166, 172, 178, 184, 190, 196, 202, 208, 214, 220, 124, 130, 136, 142, 148, 154, 160]
Target  : 154
Target 154: Modified binary -> index=15, comparisons=4 | Linear -> index=15, comparisons=16
Missing target 155: Modified binary -> index=-1, comparisons=5 | Linear -> index=-1, comparisons=17
```

```python
#[EXERCISE 1]
def linear_search(data, target):
    c = 0
    for i, v in enumerate(data):
        c += 1
        if v == target:
            return i, c
    return -1, c

def rotated_binary_search(a, target):
    low, high, c = 0, len(a) - 1, 0
    while low <= high:
        mid = (low + high) // 2
        c += 1
        if a[mid] == target:
            return mid, c
        if a[low] <= a[mid]:
            if a[low] <= target < a[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:
            if a[mid] < target <= a[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1, c

# 10 : SEED DIGIT
```

[6]

Python